Cahier des charges

🚀 PHASE 1 — Préparation de l'environnement

1️⃣ Vérifier version PostgreSQL

Tâche :

- Vérifier compatibilité avec pgvector
- Confirmer support index ivfflat ou hnsw

Objectif :

Garantir que la base supporte la recherche vectorielle performante.

2️⃣ Activer les extensions nécessaires

Tâche :

- Activer pgvector
- Activer pg_trgm
- Activer unaccent (recommandé)

Objectif :

Disposer des briques techniques nécessaires au matching hybride.

🏗️ PHASE 2 — Structuration des données

3️⃣ Ajouter colonnes techniques à ingredients

Tâche :

- Ajouter search_vector (tsvector)
- Ajouter embedding (vector(n))

Objectif :

Préparer la table à supporter les deux types de recherche :

- linguistique
- sémantique

4️⃣ Mettre en place la normalisation texte

Tâche :

- Standardiser name (lowercase, trim, unaccent)

Objectif :

Éviter que des variations de casse ou d'accents biaisent l'indexation.

⇒ deja fait normalement dans le script de preprocessing, pas besoin de le refaire en sql

🧠 PHASE 3 — Indexation initiale

5️⃣ Générer les search_vector

Tâche :

- Construire les vecteurs full-text pour tous les ingrédients existants

Objectif :

Permettre le filtrage lexical rapide et le ranking linguistique.

6️⃣ Générer les embeddings

Tâche :

- Choisir modèle sentence-transformer
- Encoder chaque ingrédient

- Stocker le vecteur dans embedding

Objectif :

Créer la représentation sémantique dense des ingrédients.

7️⃣ Créer les index

Tâche :

- Index GIN sur search_vector
- Index ANN (ivfflat ou hnsw) sur embedding
- Index trigram sur name

Objectif :

Garantir des requêtes rapides en production.

🔬 PHASE 4 — Construction du pipeline de matching

8️⃣ Définir le pipeline runtime

Tâche :

Implémenter la séquence :

1. Normalisation entrée IA
2. Génération embedding requête
3. Recherche full-text
4. Recherche vectorielle
5. Fusion candidats

Objectif :

Créer la mécanique hybride contrôlée.

9️⃣ Implémenter le scoring combiné

Tâche :

- Récupérer ts_rank
- Récupérer distance vectorielle
- Calculer score_final

Objectif :

Unifier lexical + sémantique en un score unique exploitable.

🔟 Définir un seuil de validation

Tâche :

- Déterminer un threshold (ex: 0.65–0.75)
- Définir règles :
  - auto-match
  - validation humaine
  - rejet
-

Objectif :

Éviter faux positifs nutritionnels.

🧪 PHASE 5 — Calibration (critique)

1️⃣1️⃣ Construire un dataset de test

Tâche :

- Collecter 200–500 exemples réels IA
- Annoter le “bon” ingrédient

Objectif :

Mesurer objectivement la précision.

1️⃣2️⃣ Comparer les approches

Tâche :

Mesurer :

- FTS seul
- Vector seul
- Hybride

Objectif :

Valider que l'hybride apporte réellement un gain.

1️⃣3️⃣ Ajuster les pondérations

Tâche :

- Ajuster poids lexical vs sémantique
- Ajuster paramètres ANN (lists, probes)

Objectif :

Optimiser précision et latence.

⚙️ PHASE 6 — Robustesse & Sécurité

1️⃣4️⃣ Gérer les cas ambigus

Tâche :

Définir logique si :

- 2 scores proches
- score trop bas
- aucun match

Objectif :

Contrôler le risque métier.

1️⃣5️⃣ Mettre en place logging

Tâche :

- Logger requête IA

- Score final
- Ingrédient matché
- Décision

Objectif :

Permettre audit et amélioration continue.

📈 PHASE 7 — Monitoring performance

1️⃣6️⃣ Analyser latence

Tâche :

- Mesurer temps FTS
- Mesurer temps vector
- Analyser EXPLAIN ANALYZE

Objectif :

Maintenir latence acceptable (<100–150ms selon volume).

1️⃣7️⃣ Ajuster paramètres index vectoriel

Tâche :

Optimiser :

- lists
- probes
- ou paramètres HNSW

Objectif :

Compromis précision / rapidité.

🔄 PHASE 8 — Industrialisation

1️⃣8️⃣ Automatiser l'indexation

Tâche :

Lorsqu'un nouvel ingrédient est ajouté :

- recalcul embedding
- recalcul search_vector

Objectif :

Maintenir cohérence du système.

1️⃣9️⃣ Mettre en place versioning modèle

Tâche :

- Stocker version du modèle embedding
- Permettre re-indexation future

Objectif :

Préparer évolutions IA.

🎯 Résultat attendu à la fin

- Matching robuste
- Faible taux de faux positifs nutritionnels
- Latence maîtrisée
- Système mesurable
- Évolutif